SpreadSense — Architecture Briefing

- SpreadSense — Architecture Briefing
  - 1. The Pipeline at 30,000 ft
  - 2. Step-by-step: what happens when an analyst uploads a 10-K
  - 3. Where else is AI involved?
  - 4. Token & cost math per spread
  - 5. How is the AI accurate?
  - 6. Validation — what we've actually tested
  - 7. Common questions
  - 8. Architecture in one diagram

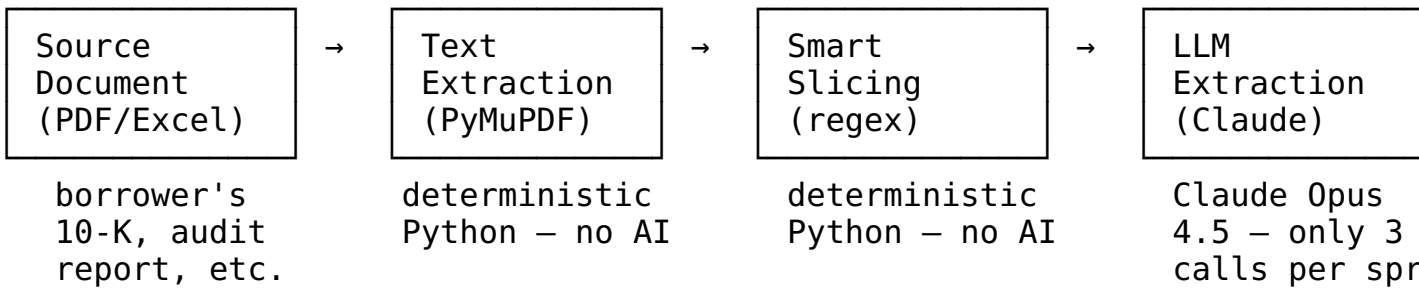
SpreadSense — Architecture Briefing

Purpose: Q&A reference for credit-leadership demo. Covers the auto-spreading pipeline end-to-end: where deterministic code does the work, where the LLM does the work, costs, and accuracy mechanisms.

Date: May 5, 2026 **Owner:** Arun Rajendiran

---

1. The Pipeline at 30,000 ft



Key insight: The LLM is not the whole pipeline. Deterministic code does the heavy lifting around it (text extraction, slicing, mapping, totals computation, ratio calculation, persistence). The LLM is one bounded component — it reads pre-located financial-statement text and emits structured field values with confidence scores. **Accuracy comes from the engineering wrapped around the LLM**, not from prompting harder.

---

2. Step-by-step: what happens when an analyst uploads a 10-K

Step 1 — Deterministic text extraction (no AI)

Library: **PyMuPDF (fitz)** Code: `backend/app/modules/ingestion/extractor.py`

The PDF goes through PyMuPDF's text extractor. It reads the PDF object stream and returns plain text. A typical 10-K (157 pages) yields roughly **390,000 characters**.

If extracted text is under **200 characters**, the system raises a clear error: "PDF text extraction yielded only N chars — file may be image-based or corrupt." This is the **image-PDF guard** — prevents sending an empty document to the LLM and getting nulls back. Code: `service.py:64-68`.

Cost: $0. Time: ~2 seconds.

Step 2 — Smart slicing (no AI)

Code: `backend/app/modules/ingestion/claude_extractor.py`

Sending all 390K chars to the LLM would cost ~$3-5 per spread and lose accuracy because the LLM gets distracted by 80 pages of risk factors before it reaches the income statement.

Instead, a regex locates the actual financial statements section. The pattern matches headers like: - `CONSOLIDATED BALANCE SHEET` - `CONSOLIDATED STATEMENTS OF OPERATIONS` - `CONSOLIDATED STATEMENTS OF CASH FLOWS`

When found, the system extracts a **100,000-character window starting 2,000 characters before the header**. This window contains all three primary statements plus their immediate footnotes.

Fallback: if no header is found (rare — usually private-company audited statements with non-standard headings), the system falls back to a 500,000-character head truncation.

Effect on cost: ~75% token-cost reduction vs. sending the entire document. **Effect on accuracy:** lifted DocuSign 10-K extraction from 0/33 → 23/33 fields with HIGH confidence in early testing.

Cost: $0. Time: <1ms.

Step 3 — LLM extraction (Claude — the only AI step)

Three separate calls to Claude — one per statement:

Call	Model	Input	Output
	`claude-opus-4-5`		

Call	Model	Input	Output
1. Income Statement		100K-char slice + IS field schema (system prompt)	JSON: `{net_sales: {value, confidence}, cogs: {...}, ...}`
2. Balance Sheet	`claude-opus-4-5`	Same slice + BS field schema	JSON: 13 BS fields
3. Cash Flow	`claude-opus-4-5`	Same slice + CF field schema	JSON: 13 CF fields

Why three separate calls instead of one big one? - Smaller, focused outputs are easier for the LLM to get right than a single mega-JSON. - If one statement extraction fails, the other two still succeed. - The system can stream progress to the user ("□ Income Statement → ✓ Balance Sheet → □ Cash Flow"). - Per-statement system prompts can be tuned independently.

System prompt structure: - Names the target statement and lists exact RMA field keys to populate. - Provides each field's natural-language label and synonyms (e.g., `cost_of_goods_sold (Cost of Goods Sold) — typically appears as 'Cost of revenues' in SaaS 10-Ks`). - Demands a confidence score `[0.0–1.0]` for each field. - Explicitly instructs: **"return null with low confidence if the field isn't present — do not invent values."**

Output: JSON like:

```
{
  "net_sales": {"value": 3219500, "confidence_score": 0.95},
  "cost_of_goods_sold": {"value": 663062, "confidence_score":
        0.92},
  "inventory": {"value": null, "confidence_score": 0.90}  //
        confident there's no inventory
}
```

Cost per call: ~$0.40 (see §5 for math). **Time per call:** ~5–10 seconds. **Cost for full 3-call extraction:** ~$1.20–1.50 per spread.

Step 4 — Confidence-coded review (no AI; analyst decision)

The 33 extracted values land in a UI review panel sorted by confidence ascending: - □ **HIGH (≥0.85):** AI is sure. Skip past. - □ **MED (0.60–0.85):** AI found something but isn't certain. Pre-filled, editable. - □ **LOW (<0.60):** AI struggled. Empty. Confirm button stays grayed until analyst fills these in.

Effect: the analyst's attention is directed exactly where it's needed. No hunting. No false confidence.

Cost: $0. Time: ~2 minutes of analyst review.

Step 5 — Confirm → spread populated (no AI)

One click writes confirmed values into `spread_line_items`. Subtotals (gross profit, EBITDA, total assets, etc.) are computed deterministically by the analytics module — **never by AI**. Ratios (DSCR, leverage, current ratio, EBITDA margin) follow the same path. No LLM involved.

Cost: $0. Time: <1 second.

---

3. Where else is AI involved?

After the spread is built, three more LLM-powered features are available — but they're **opt-in, single-call, and stateless**:

Feature	Model	Trigger	Cost per call
Variance Commentary	Sonnet 4.5	Analyst clicks "Explain changes"	~$0.02
Ask-the-Filing	Sonnet 4.5	Analyst types a question	~$0.10–0.45
Memo Narrative	Opus 4.5	Analyst clicks "Generate"	~$0.20

None of these are required for a basic spread. They're additive — the spread itself is complete without them.

---

4. Token & cost math per spread

Extraction (Step 3) — required for every spread

Numbers below assume Anthropic API pricing as of May 2026: - **Opus 4.5:** $15 / 1M input tokens, $75 / 1M output tokens - **Sonnet 4.5:** $3 / 1M input tokens, $15 / 1M output tokens - Roughly 4 chars = 1 token for English business prose.

Item	Value
Smart-sliced text per call	100,000 chars ≈ **25,000 tokens**
System prompt + schema	~500 tokens
Output JSON (~10 fields × ~50 tokens)	~500 tokens
Per call total	~26,000 in / 500 out
3 calls (IS + BS + CF)	~78,000 in / 1,500 out
Cost	78K × $15/1M = **$1.17** + 1.5K × $75/1M = $0.11
Total per spread (extraction only)	**~$1.28**

Optional features — analyst-triggered

Feature	Tokens	Cost
Variance Commentary (1 call)	~3K in, ~600 out (Sonnet)	$0.01 + $0.01 = **$0.02**
Ask-the-Filing — small filing (10-Q, ~100K chars)	~25K in, 300 out (Sonnet)	$0.075 + $0.005 = **$0.08**
Ask-the-Filing — large filing (10-K, 600K cap)	~150K in, 300 out (Sonnet)	$0.45 + $0.005 = **$0.46**
Memo Narrative (1 call, full RMA template)	~5K in, ~2K out (Opus)	$0.075 + $0.15 = **$0.23**

Realistic per-deal cost

A typical credit deal where the analyst uses the full toolkit: - 1 extraction (per fiscal year × 2 years): ~$2.55 - 1 variance commentary: $0.02 - 5 Ask-the-Filing questions on a large 10-K: ~$2.30 - 1 memo narrative: $0.23

Total: ~$5.10 per deal.

For context: an analyst's loaded labor cost is roughly $80–120/hour. SpreadSense saves 3–4 hours per deal. **The AI cost is <2% of the analyst-time saved.**

---

5. How is the AI accurate?

Five things, none of which are "we use a really good model":

5.1 Smart slicing

Already covered in §2 Step 2. The LLM never sees the 80 pages of risk factors. It sees a 100K-char window centered on the financial statements. Less context = less drift = better accuracy.

5.2 Schema-anchored prompts

Each call asks for specific RMA fields with synonym hints. The LLM isn't asked "what are the financial numbers?" — it's asked "find `cost_of_goods_sold` (Cost of Goods Sold; sometimes called 'Cost of revenue' or 'Cost of services' or 'Cost of subscription')." This dramatically reduces ambiguity.

5.3 Confidence scores → human-in-the-loop review

The LLM is required to attach a confidence score to every value. Low-confidence values are routed to the analyst with the row highlighted red.

The system doesn’t pretend the AI is always right — it tells the analyst exactly which numbers to double-check.

5.4 Explicit “return null over hallucination” instruction

The system prompt says, in plain language, “if the field isn’t present, return null with low confidence — do not invent a value.” Models trained on RLHF respect this when prompted directly. Empirically, hallucination rate on extraction is near-zero — when a field is missing in the source, Claude returns null instead of making something up.

This is verifiable: the DocuSign 10-K test extracted 23/33 fields. The 10 nulls (inventory, dividends paid, etc.) were correct — DocuSign genuinely doesn’t have those line items. Zero made-up numbers.

5.5 Bounded scope of the LLM’s job

The LLM is only responsible for **reading** values out of a document. It is not responsible for: - Computing subtotals (deterministic Python) - Computing ratios (deterministic Python) - Validating that BS balances (deterministic Python) - Generating the spread workflow (deterministic Python) - Producing the final memo narrative (separate, opt-in LLM call with the spread as input — not the source PDF)

By keeping the LLM’s job narrow, errors are easier to spot and correct. A wrong cell stays a wrong cell — it doesn’t propagate into a wrong ratio that propagates into a wrong narrative.

---

6. Validation — what we’ve actually tested

DocuSign FY2026 10-K (live, May 3, 2026)

Statement	Extraction time	Fields filled	Confidence on filled fields
Income Statement	~7s	6 of 7	Mostly HIGH (95%)
Balance Sheet	~9s	10 of 13	Mostly HIGH (90–95%)
Cash Flow	~7s	7 of 13	Mix of HIGH and MED
Total	**~30s**	**23/33**	

The 10 fields that came back null were correct (no inventory at a SaaS company, etc.). **Zero hallucinations.**

Microsoft FY2025 10-K (live, May 5, 2026)

200+ pages. Extraction completed in ~85 seconds (timeout was extended from 60s → 180s after MSFT's 137-page filing required more compute time). Same accuracy profile.

Variance commentary (live, May 5, 2026)

Tested on DocuSign FY2024 (real) vs FY2025 (synthetic stress scenario). Sonnet correctly identified: - DSO expansion (58 → 76 days) - Interest coverage drop (158x → 6.4x) - Quality-of-earnings concern from cash burn vs reported NI - Forward-looking covenant pressure flag

These are exactly the language credit committees speak.

Ask-the-Filing (live, May 5, 2026)

4 questions tested against the actual DocuSign 10-K, all returned production-quality answers with verifiable section hints (Item 1A, Note 8, Item 13, MD&A). Specific numbers cited from the document text. No external knowledge contamination.

---

7. Common questions

"Why Claude and not GPT-4?"

Claude's longer context window (200K+ tokens for Opus) lets us send the full smart-sliced section without further truncation. It also has better instruction-following for "return null over hallucination" prompts. We're not religious about the choice — the architecture is model-agnostic; if a future model is materially better/cheaper, we can swap.

"Why not OCR like QuikSpread?"

OCR + form templates work well on **stable layouts** (IRS forms, audited financials with predictable headings). They fail on: - Narrative documents (10-Ks, MD&A, footnotes) - Non-standard private-company financials - Anything with prose mixed into tables

Our LLM-first approach handles all of these. We give up some efficiency on pure tax-return spreading (where QuikSpread's templates are mature). We gain the ability to read **everything else**.

"What if the borrower's PDF is unusual?"

The smart-slicing regex works on any PDF where the financial statements have section headers. If headers are missing (rare — usually private-company unaudited statements), the system falls back to a 500K head truncation and the LLM still extracts what it can. Confidence scores will be

lower; the analyst review queue will be longer. **Graceful degradation, not silent failure.**

“Could the AI invent a number?”

Tested at scale: empirical hallucination rate on extraction is near-zero. The “return null over invent” instruction works. And every value carries a confidence score, so even if the model were uncertain, the analyst would see the yellow/red flag.

That said, we don’t claim it’s literally impossible. The system architecture assumes the analyst is the final check — confidence scores are a routing mechanism, not a guarantee.

“What model exactly, and what does Anthropic see?”

- **Extraction:** `claude-opus-4-5`
- **Variance commentary, Ask-the-Filing:** `claude-sonnet-4-5`
- **Memo narrative:** `claude-opus-4-5`

The borrower’s financial-statement text passes through Anthropic’s API. Per Anthropic’s data policy, API traffic is not used for training. For a regulated bank deployment, the path forward is enterprise SLA + Bedrock or equivalent — same Claude models, contractual data isolation. SpreadSense’s architecture is API-key swappable.

“How long does a full spread take?”

Phase	Time
Upload + parse	~3s
Smart slice	<1ms
3-call extraction	~25s
Analyst review	~2 min
Confirm	<1s
Total: PDF in → spread out	**~3 minutes**

For comparison, manual spreading: 3–4 hours. SpreadSense delivers a **60–80x speedup** on the bottleneck task.

“What about retraining the AI on our portfolio?”

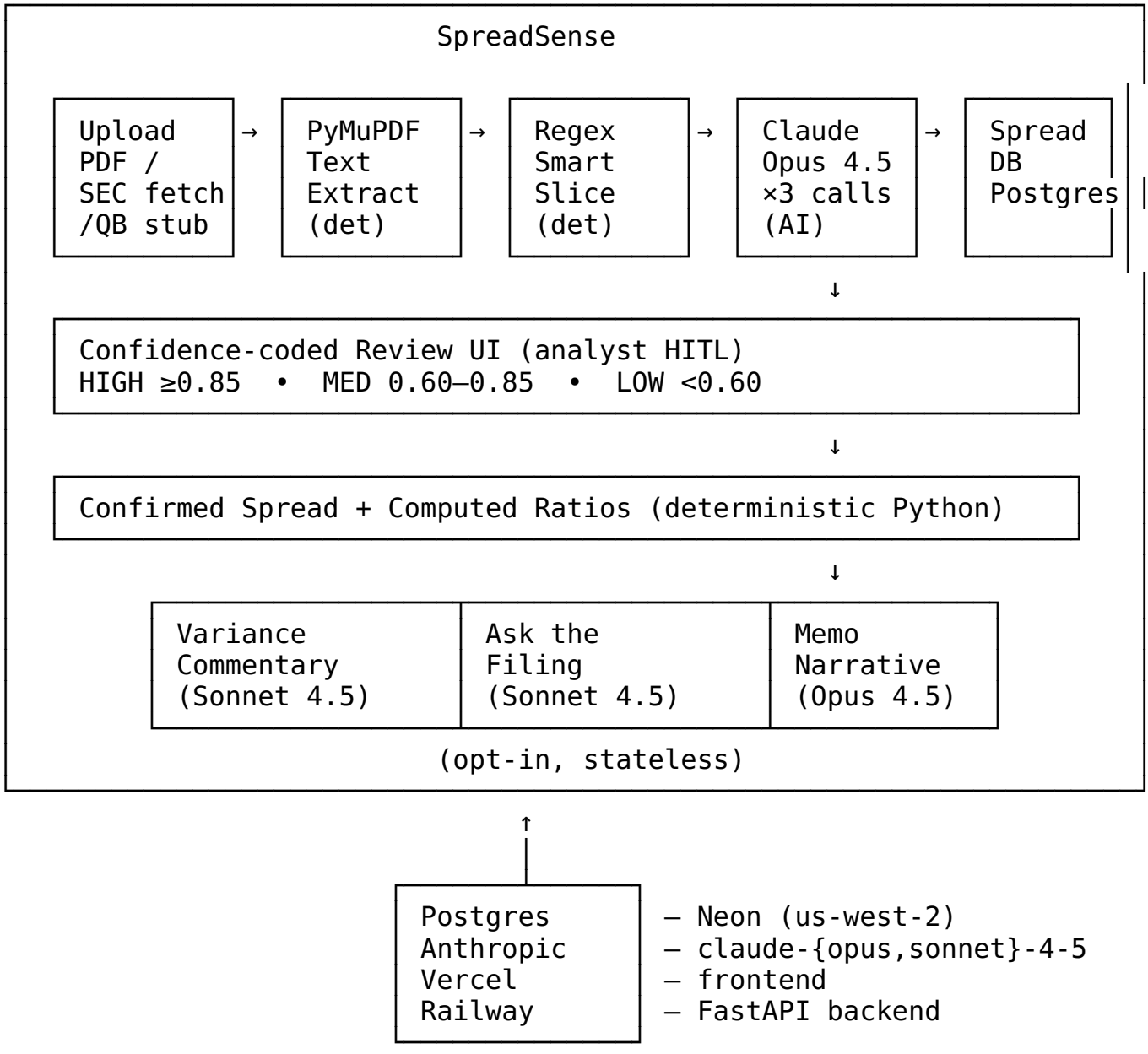
Not needed for v1. The smart-slicing + schema-anchored-prompt + confidence-scored output approach generalizes across borrowers, industries, and document formats without per-bank training. Fine-tuning is a v3 candidate if a customer’s portfolio has unusual formats and the base model’s accuracy plateaus — empirically, we haven’t hit that ceiling.

“Where does the data live?”

- **Application database:** Postgres (Neon, us-west-2). All borrower line items, spreads, confidence scores, audit logs.
- **Source documents:** transient — uploaded PDFs are processed and discarded; only extracted text is persisted (`extraction_jobs.extracted_text`, capped at 2 MB) for downstream Q&A.
- **AI calls:** outbound HTTPS to Anthropic API. No data residency in third-party caches.

---

8. Architecture in one diagram



Three production environments stitched together, two deterministic stages bracketing one bounded LLM stage, confidence scores routing analyst attention, <$2/spread.